Часть 1.

Object Explorer — это панель в SQL Server Management Studio (SSMS) для просмотра и управления структурой сервера баз данных в виде дерева. В ней есть:

- сервер (экземпляр SQL Server) и информация о его версии;
- Databases — список баз данных;
- Security — логины, роли сервера, учётные данные;
- Server Objects — связанные серверы, эндпоинты, триггеры сервера;
- Replication, Management, SQL Server Agent (задания, оповещения).

Внутри каждой базы данных дерево раскрывается на подпапки:

- Tables — таблицы базы данных;
- Views — представления;
- Synonyms;
- Programmability — хранимые процедуры, функции, триггеры базы данных;
- Security — пользователи, роли, схемы конкретной базы;
- Storage — файловые группы, полнотекстовые каталоги.

Окно New Query нужно для написания и выполнения произвольных SQL-запросов (SELECT, INSERT, UPDATE, DDL-команды создания объектов и т.д.) напрямую к выбранной базе данных, это рабочая область для ручного написания и запуска T-SQL кода, в отличие от Object Explorer, который даёт только визуальный обзор структуры.

Часть 2.

Какие настройки базы данных можно изменить при создании?
При создании базы данных (через диалог New Database в SSMS или командой CREATE DATABASE) можно настроить:

- Имя базы данных (Database name);
- Владельца (Owner);
- Файлы данных и журнала — имя логического файла, путь размещения, начальный размер, параметры автоувеличения (Autogrowth) и максимальный размер;
- Filegroups — файловые группы для распределения данных;
- Collation (параметры сортировки/кодировки);
- Recovery model (модель восстановления: Full, Simple, Bulk-logged);
- Compatibility level (уровень совместимости с версией SQL Server);
- прочие параметры Options — например, Auto Close, Auto Shrink, режим доступа (Restrict Access) и др.

Отличие удаления базы от переименования

- Удаление (DROP DATABASE) — полностью и безвозвратно уничтожает базу данных: все её файлы данных и журнала удаляются с диска, вместе со всеми таблицами, данными, объектами. Восстановить можно только из резервной копии.
- Переименование (ALTER DATABASE ... MODIFY NAME) — база данных, все её объекты и данные остаются нетронутыми, меняется только её логическое имя, под которым она отображается и к которому обращаются по имени. Физические файлы на диске при этом не переименовываются автоматически.

Почему важно правильно выбирать имя базы данных

- Имя используется во всех запросах и строках подключения — при неудачном или неоднозначном имени усложняется сопровождение кода и увеличивается риск ошибок;
- влияет на читаемость и понятность структуры при работе в команде, особенно если баз много;
- переименование существующей базы может нарушить работу приложений, скриптов и заданий, которые обращаются к ней по старому имени, поэтому лучше сразу выбрать корректное имя;

- имя не должно конфликтовать с зарезервированными словами и должно соответствовать принятым в проекте соглашениям об именовании.

Какие типы данных вы использовали для колонок и почему?

- INT / BIGINT — для целочисленных значений и ключей (ID), если значения не превышают диапазон — используют INT, для больших объёмов — BIGINT;
- VARCHAR(n) / TEXT — для строк переменной длины (имена, описания); VARCHAR с ограничением длины — когда известен разумный максимум, TEXT — когда длина непредсказуема;
- CHAR(n) — для строк фиксированной длины (например, коды из фиксированного числа символов);
- DATE / TIMESTAMP — для дат и даты со временем;
- BOOLEAN — для логических флагов (да/нет);
- DECIMAL/NUMERIC — для точных числовых значений (деньги), где недопустима погрешность float;
- REAL/FLOAT — для чисел с плавающей точкой, где точность менее критична.

Для чего нужны ключи (PRIMARY KEY и FOREIGN KEY)?

- PRIMARY KEY (первичный ключ) — уникально идентифицирует каждую строку таблицы. Гарантирует отсутствие дублирующихся записей и NULL-значений в ключевом поле, служит точкой привязки для связей с другими таблицами.
- FOREIGN KEY (внешний ключ) — устанавливает связь между таблицами, ссылаясь на PRIMARY KEY другой таблицы. Обеспечивает ссылочную целостность: не позволяет вставить в дочернюю таблицу значение, которого нет в родительской, и контролирует поведение при удалении/изменении связанных записей (CASCADE, RESTRICT, SET NULL и т.д.).

Вместе они реализуют реляционные связи между таблицами (один-ко-многим, многие-ко-многим через связующую таблицу) и защищают базу от противоречивых данных.

Как изменение структуры таблицы влияет на данные?

- Добавление колонки — обычно безопасно; новым полям присваивается NULL или значение по умолчанию (DEFAULT) для уже существующих строк.
- Удаление колонки — необратимо удаляет все данные в этом столбце для всех строк.
- Изменение типа данных колонки (ALTER COLUMN) — может привести к ошибке или потере данных, если существующие значения несовместимы с новым типом (например, из VARCHAR в INT при наличии нечисловых строк); СУБД пытается выполнить неявное приведение типов, но не всегда успешно.
- Добавление ограничения NOT NULL — завершится ошибкой, если в столбце уже есть NULL-значения.
- Добавление/удаление PRIMARY KEY или FOREIGN KEY — может завершиться ошибкой при наличии дублей (для PK) или "осиротевших" ссылок (для FK), либо потребует предварительной очистки данных.
- Переименование колонки/таблицы — сами данные не меняются, но может нарушить работу запросов, представлений (views) и приложений, которые ссылаются на старое имя.

Чем отличается добавление данных от их изменения?

- Добавление (INSERT) — создаёт новую строку (запись) в таблице. Все существующие данные остаются нетронутыми, просто увеличивается количество строк.
- Изменение (UPDATE) — меняет значения в уже существующих строках по заданному условию (WHERE). Количество строк в таблице не меняется, но меняется их содержимое. Если не указать WHERE, UPDATE изменит значения во всех строках таблицы — частая и опасная ошибка.

Разница по сути: INSERT работает с новыми записями, UPDATE — с уже существующими.

Как фильтры и сортировка помогают работать с таблицей?

- Фильтрация (WHERE / HAVING) — позволяет выбрать только те строки, которые соответствуют определённому условию, вместо просмотра всей таблицы целиком. Это ускоряет работу с данными и позволяет анализировать конкретные подмножества (например, найти пользователей из одного города или заказы за конкретный месяц).
- Сортировка (ORDER BY) — упорядочивает результат по одному или нескольким столбцам (по возрастанию ASC или убыванию DESC), что облегчает восприятие данных, поиск максимумов/минимумов, построение рейтингов и т.д.

Как связи между таблицами влияют на видимые данные?

- Связи (через FOREIGN KEY) определяют, какие данные можно объединить при выполнении запроса с помощью JOIN — например, соединить таблицу заказов с таблицей клиентов, чтобы увидеть имя клиента рядом с его заказом, хотя эти данные физически хранятся в разных таблицах.
- Тип связи влияет на то, что покажет запрос:
    - INNER JOIN — покажет только те строки, у которых есть совпадение в обеих таблицах;
    - LEFT JOIN — покажет все строки из левой таблицы, даже если для них нет связанной записи в правой (будет NULL);
- Ссылочная целостность (FOREIGN KEY constraint) не даёт удалить или изменить запись, если на неё ссылаются другие таблицы (если не настроено CASCADE), что защищает от появления "битых" связей, когда данные в одной таблице ссылаются на несуществующие данные в другой.
- Итог: связи не хранят видимые данные сами по себе, но определяют, какие комбинации данных можно получить и как согласованно они будут отображаться при запросах, объединяющих несколько таблиц.
-